

```
import itertools
import numpy as np
```

```
def bipolar_activation(net):
    return 1 if (net >= 0) else -1
```

```
def unipolar_activation(net):
    return 1 if (net >= 0) else 0
```

```
def predict(X, weights, type):
    net = np.dot(X, weights)
    if type == 'unipolar':
        return unipolar_activation(net)
    elif type == 'bipolar':
        return bipolar_activation(net)
```

```
def train(X, y, weights, type, epochs=500, c=0.5):
    for epoch in range(epochs):
        loss = 0
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights, type)
            r = yi - y_pred
            loss += abs(r)
            delta_w = c*r*Xi
            weights += delta_w

        print(f'Weights after epoch {epoch}: {weights}')
        if loss == 0:
            break

    print('Learned weights : ', weights)
    weights = weights.reshape((n + 1, 1))
    test(X, y, weights, type)
```

```
def test(X, y, learned_weights, type):
    nets = np.dot(X, learned_weights).flatten()
    print('Actual Values : ', y)
    if type == 'unipolar':
        y_pred = np.array([1 if net >= 0 else 0 for net in nets])
        print('Predicted Values : ', y_pred)
    else:
        y_pred = np.array([1 if net >= 0 else -1 for net in nets])
        print('Predicted Values : ', y_pred)
```

```
n = int(input('Enter number of bits : '))
X = np.array([list(i) + [1] for i in itertools.product([0,1], repeat = n)])

weights = input(f'Enter initial {n} weights and 1 bias : ')
weights = np.array([float(weight) for weight in weights.split()], dtype='longdouble')
print()
```

```
↻ Enter number of bits : 2
Enter initial 2 weights and 1 bias : 1 1 -1.5
```

```
# 1) AND GATE UNIPOLAR
print('---- AND GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'unipolar')
```

```
↻ ---- AND GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.  1. -1.5]
Learned weights : [ 1.  1. -1.5]
Actual Values : [0 0 0 1]
Predicted Values : [0 0 0 1]
```

```
# 2) OR GATE UNIPOLAR
print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = 0
train(X, y, weights.copy(), 'unipolar')
```

```
↻ ---- OR GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.  1.5 -1. ]
```

```

Weights after epoch 1: [ 1.   1.5 -1. ]
Learned weights : [ 1.   1.5 -1. ]
Actual Values : [0 1 1 1]
Predicted Values : [0 1 1 1]

```

3) NOR GATE UNIPOLAR

```

print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([0]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'unipolar')

```

```

---- NOR GATE USING PERCEPTRON ----
Weights after epoch 0: [ 0.5  0. -2. ]
Weights after epoch 1: [ 0.5  0. -1.5]
Weights after epoch 2: [ 0.5  0. -1. ]
Weights after epoch 3: [ 0.  0. -1.]
Weights after epoch 4: [ 0.  0. -0.5]
Weights after epoch 5: [ 0. -0.5 -0.5]
Weights after epoch 6: [-0.5 -0.5 -0.5]
Weights after epoch 7: [-0.5 -0.5 0. ]
Weights after epoch 8: [-0.5 -0.5 0. ]
Learned weights : [-0.5 -0.5 0. ]
Actual Values : [1 0 0 0]
Predicted Values : [1 0 0 0]

```

4) NAND GATE UNIPOLAR

```

print('---- NAND GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[-1] = 0
train(X, y, weights.copy(), 'unipolar')

```

```

---- NAND GATE USING PERCEPTRON ----
Weights after epoch 0: [ 0.5  0.5 -1.5]
Weights after epoch 1: [ 0.   0.5 -1. ]
Weights after epoch 2: [ 0.   0. -0.5]
Weights after epoch 3: [-0.5 -0.5 -0.5]
Weights after epoch 4: [-1.  -0.5 0. ]
Weights after epoch 5: [-1.  -0.5 0.5]
Weights after epoch 6: [-1.  -1.  0.5]
Weights after epoch 7: [-1.  -0.5 1. ]
Weights after epoch 8: [-1.  -0.5 1. ]
Learned weights : [-1.  -0.5 1. ]
Actual Values : [1 1 1 0]
Predicted Values : [1 1 1 0]

```

5) AND GATE BIPOLAR

```

print('---- AND GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[-1] = 1
train(X, y, weights.copy(), 'bipolar')

```

```

---- AND GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.   1. -1.5]
Learned weights : [ 1.   1. -1.5]
Actual Values : [-1 -1 -1 1]
Predicted Values : [-1 -1 -1 1]

```

6) OR GATE BIPOLAR

```

print('---- OR GATE USING PERCEPTRON ----')
y = np.array([1]*(2**n))
y[0] = -1
train(X, y, weights.copy(), 'bipolar')

```

```

---- OR GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.   2. -0.5]
Weights after epoch 1: [ 1.   2. -0.5]
Learned weights : [ 1.   2. -0.5]
Actual Values : [-1 1 1 1]
Predicted Values : [-1 1 1 1]

```

7) NOR GATE BIPOLAR

```

print('---- NOR GATE USING PERCEPTRON ----')
y = np.array([-1]*(2**n))
y[0] = 1
train(X, y, weights.copy(), 'bipolar')

```

```

---- NOR GATE USING PERCEPTRON ----
Weights after epoch 0: [ 1.   0. -1.5]
Weights after epoch 1: [ 0.   0. -1.5]
Weights after epoch 2: [ 0.   0. -0.5]
Weights after epoch 3: [ 0.  -1. -0.5]
Weights after epoch 4: [-1. -1. -0.5]
Weights after epoch 5: [-1. -1.  0.5]

```

```
Weights after epoch 6: [-1. -1.  0.5]
Learned weights : [-1. -1.  0.5]
Actual Values : [ 1 -1 -1 -1]
Predicted Values : [ 1 -1 -1 -1]
```

```
# 8) NAND GATE BIPOLAR
```

```
print('---- NAND GATE USING PERCEPTRON ----')
```

```
y = np.array([1]*(2*n))
```

```
y[-1] = -1
```

```
train(X, y, weights.copy(), 'bipolar')
```



```
---- NAND GATE USING PERCEPTRON ----
Weights after epoch 0: [ 0.  0. -1.5]
Weights after epoch 1: [-1.  0. -0.5]
Weights after epoch 2: [-1. -1.  0.5]
Weights after epoch 3: [-2. -1.  0.5]
Weights after epoch 4: [-2. -1.  1.5]
Weights after epoch 5: [-2. -2.  1.5]
Weights after epoch 6: [-2. -1.  2.5]
Weights after epoch 7: [-2. -1.  2.5]
Learned weights : [-2. -1.  2.5]
Actual Values : [ 1  1  1 -1]
Predicted Values : [ 1  1  1 -1]
```